

Persistent State and Becoming

When Memory Becomes Operational History

Zawwar Sami

Independent Researcher, Canada

ORCID: [0009-0004-5819-3017](https://orcid.org/0009-0004-5819-3017)

7 October 2026 · Version 1.5 · Author’s manuscript

Abstract An AI system can keep records for months and still have no usable past. It may retrieve the right sentences yet fail to know what was revoked, what failed, what was later verified, or why the present state differs from an earlier one. I ask where that boundary lies. I distinguish five layers: archive, context, persistent state, operational history, and becoming. My claim is narrow: records become operational history when relations among events—including order, supersession, verified outcome, and provenance—change what the deployed system should do now. I turn that claim into a representation-adequacy test and a counterfactual pre/post-state test for a limited form of system-level becoming. The latter requires a durable, attributable change in later behavior; it does not require model-weight updates and does not imply consciousness or numerical identity. I place the account within recent work on long-term agent memory, use a publication workflow to show how identical surviving records can encode different practical pasts, and specify a controlled M1–M4 evaluation with relational ablations. I do not report a benchmark result. What I am trying to sharpen is the difference between record survival, continuity of work, and adaptation caused by retained experience.

Keywords: artificial agents; persistent state; long-term memory; episodic memory; temporal reasoning; agent history; continual learning; provenance; model replacement

1 A file can survive and still have no past

Imagine an assistant that has worked with me on a manuscript for several days. On Monday the draft is marked ready. On Tuesday a citation audit finds that one central reference cannot be verified, so the release is blocked. On Wednesday the reference is replaced, the claim is narrowed, and the manuscript passes the audit. Three statements now survive in memory: *ready*, *blocked*, and *ready*. If I ask on Thursday whether the paper can be posted, a database containing all three statements has not yet solved the problem. The system has to know which state followed which, what caused each transition, and whether the later state actually superseded the earlier one.

This is easy to overlook because “memory” is asked to do several jobs at once. It can mean a stored conversation, a retrieved fact, a current task variable, a record of previous events, or an adaptation produced by those events. The differences matter more as language-model systems operate across long intervals. LongMemEval, for example, evaluates not only information extraction but multi-session reasoning, temporal reasoning, knowledge updates, and abstention [14]. Recent memory architectures likewise give chronology a more explicit role: SegTreeMem preserves temporal order during

memory construction [2], while ARTEM is designed around spatial-temporal episodic retrieval [13]. Systems work now treats agent memory as a stateful write-and-read workload in which information is persistently constructed, retrieved, and updated rather than as a passive vector store [3].

These developments motivate a narrower question than “does the agent have memory?” I ask: *when does surviving information become a history that has practical force in the present?* The distinction matters for a second reason. It is increasingly natural to say that an agent “learned” or “grew” after it accumulated memories. That language can outrun the evidence. A larger archive can coexist with identical behavior. A rewritten summary can change a future answer without increasing any underlying competence. A system can also become more cautious, more constrained, or better calibrated rather than simply more capable.

I use *becoming* in this paper in a deliberately limited operational sense: durable change in task-relevant system state or behavior that is attributable to prior events and remains available after the episode that produced it. This is not a metaphysical thesis. It does not claim that an artificial subject persists through time, or that external memory creates consciousness. It is also not a synonym for continual learning in machine learning, where adaptation often concerns learned parameters or representations and where catastrophic forgetting is a central problem [4]. My concern is the deployed system around the model and the way a past can continue to organize what happens next.

Rather than treating those layers as synonyms, I move through them one by one and ask what each one adds. The key question is whether temporal and revision relations are actually doing work. From there I separate state-mediated change from weight-level learning, then end with an evaluation protocol that could fail if the distinction is wrong. What I offer here is a vocabulary and a set of tests, not an empirical result.

2 What current memory research already establishes

There is no shortage of evidence that long-horizon agents need mechanisms beyond a single prompt. The important task is therefore not to rebrand persistent memory as a new idea, but to identify a gap inside an increasingly mature design space.

Surveys of LLM-agent memory describe a field that already spans memory formation, storage, retrieval, reflection, and evaluation [16]. Generative Agents used a memory stream together with reflection and planning to support believable behavior over simulated time [5]. Reflexion stored linguistic feedback from prior attempts and reused it on later trials, showing one route by which experience can alter subsequent problem solving without updating the base model’s weights [7]. MemoryBank introduced long-term memory mechanisms for sustained interaction [17], and A-MEM allows newly added memories to reorganize or update attributes of earlier ones rather than treating the store as fixed [15].

Two things in this literature matter directly for my argument. Retrieval quality alone is too narrow a target. LongMemEval explicitly includes temporal reasoning and knowledge updating because a useful memory system must often distinguish what was true, what became false, and what is now current [14]. The order of experience can also carry information. Liu et al. report that preserving temporal order contributes to SegTreeMem’s performance, with permutation analyses supporting the role of chronology in long-horizon memory [2]. ARTEM similarly targets chronological awareness in episodic tasks rather than only semantic match [13]. Pink et al. argue more broadly

that episodic memory deserves explicit treatment in long-term LLM agents and organize that argument around properties needed for adaptive, context-sensitive behavior [6].

A systems perspective pushes in the same direction. Omri et al. characterize agent memory through construction, retrieval, and generation phases, emphasizing that memory is mutable state produced and maintained during ongoing workloads [3]. This is useful because it makes the write path visible. A system does not merely ask, “Which old item resembles this query?” It must also decide what to write, when to update, what to preserve, and how freshness interacts with retrieval.

There is an older parallel in temporal database research. Chakravarthy and Kim distinguish time concepts needed to preserve past states under delayed, retroactive, and corrective updates [1]. I do not claim that an AI agent is a temporal database. The narrower lesson is that storing values is not the same as representing how values changed. Once correction and revision matter, the relation between states becomes part of the information problem.

My earlier A-series papers approach adjacent issues from different directions. A1 argues that remembered personal history used on a person’s behalf should remain open to that person’s correction [8]. A2 separates copied memory from the legitimate succession of delegated assistance [9]. A3 places practical continuity in an architecture of persistent state, context reconstruction, verified outcomes, and revision rather than in a single model [10]. A4 separates model capability, execution control, and delegated authority when identifying the relevant agent boundary [11]. A5 takes the next step: even after persistent state is available, what makes it a *history* rather than a collection of context, and what evidence would show that the system has been changed by it?

3 Five layers that should not be collapsed

The five layers below are not a required software stack. One mechanism may implement several of them, and some tasks need only the first two. The point is explanatory: each layer licenses a stronger claim than the one before it.

Table 1: Five levels of persistence and the claim each can support.

Layer	What is available	What it does <i>not</i> yet establish
Archive	Records survive beyond the episode that produced them.	That any record is retrieved, current, ordered, or behaviorally consequential.
Context	Selected records are reconstructed or retrieved for a present query.	That the selection represents the current state or preserves revision relations.
Persistent state	Task-relevant status, commitments, open work, constraints, or results survive interruptions.	That the system can explain how the state arose or distinguish histories with the same surviving content.
Operational history	State changes retain sufficient temporal, supersession, outcome, and provenance relations for earlier transitions to constrain present action.	That the system improved, learned a general skill, or possesses autobiographical experience.
Becoming	Prior events produce a durable, attributable change in future task-relevant behavior or state.	Consciousness, numerical identity, human-like development, or monotonic capability gain.

An *archive* answers the survival question: is the record still there? This is already valuable. It can make an action auditable and prevent information loss. But a complete transcript can remain inert.

Context is selected information made available to the current reasoning process. Retrieval-augmented systems operate here. Context can be excellent while history is poor. A query for “publication status” might retrieve all three status statements from the opening example, perhaps ranking the first “ready” statement highest because it is semantically clean and frequently mentioned.

Persistent state is stronger. It records what is presently operative: the manuscript is blocked, a task is unfinished, a tool call failed, a constraint is active, or a decision has been superseded. A compact state can preserve practical continuity even when most raw history is omitted. This is important: operational history does not require replaying the whole archive.

Operational history adds enough structure about transitions for the system’s past to have present explanatory and control significance. The status “ready” matters differently if it followed a successful audit than if it preceded a later failure. A rejected hypothesis can remain relevant because it was rejected and because its failure changed the next move. The same words may survive in two stores while the histories differ.

Finally, *becoming* concerns change caused by experience. The system need not become more powerful. It may learn, in an ordinary engineering sense, to stop retrying a broken endpoint; it may narrow a claim after contradictory evidence; it may revoke an old plan; it may become less willing to act without verification. What matters is that a previous event leaves a durable difference in later conduct under conditions where the event itself is no longer being freshly described in the prompt.

Figure 1 summarizes the progression. The arrows should not be read as an inevitability. A system can accumulate context forever without moving to the later layers.

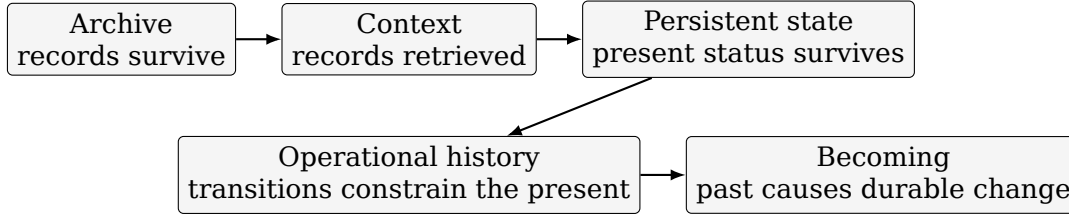


Figure 1: Persistence becomes a stronger claim only when additional relations are preserved and used. The diagram is conceptual, not a mandatory implementation pipeline.

4 A representation-adequacy test for operational history

I now make the distinction more precise. Let R_t be the set of surviving records available by time t . A retrieval mechanism produces context for query q :

$$C_t(q) = Q(R_t, q). \quad (1)$$

This is a context function. It says nothing by itself about which record is current or why.

Let S_t denote task-relevant operative state at time t . An event e_{t+1} and an observed or verified outcome o_{t+1} may update that state through

$$S_{t+1} = U(S_t, e_{t+1}, o_{t+1}). \quad (2)$$

The explicit outcome matters. Without it, “attempted upload” and “published successfully” can collapse into the same narrative. In practice U may be implemented by code,

a database transaction, an LLM-mediated updater, a human correction, or some combination. The formalism does not privilege one implementation.

I represent the operational history available at time t as

$$\mathcal{H}_t = (S_t, T_t, P_t), \quad (3)$$

where T_t contains the task-relevant transition relations among states and events, and P_t contains provenance sufficient for the current task: who or what produced an update, what evidence supported it, whether it superseded an earlier state, and where verification came from. This need not contain every prior state. A compressed state plus the transitions required for audit or reversal can be enough.

The formal point I need is a necessary adequacy condition, not a new theorem. Let h and h' be two admissible histories. Let $R(h)$ denote the multiset of surviving record content in a history, let $\phi(h)$ denote the memory representation actually exposed to the acting policy, and let $a^*(h, q)$ be the task-correct action for probe q . Suppose

$$R(h) = R(h'), \quad a^*(h, q) \neq a^*(h', q). \quad (4)$$

If $\phi(h) = \phi(h')$, a policy conditioned only on ϕ receives the same represented past in both cases and cannot reliably produce both different correct actions. The useful test is therefore not Equation 4 by itself; it is the intervention that follows from it: preserve the record content, alter only the relevant temporal or supersession relations, and ask whether the deployed system changes its action appropriately. If it does not, the representation has discarded a task-relevant part of the past.

This test is deliberately task-relative. If I ask for the spelling of a project name, order may be irrelevant. If I ask whether a revoked permission is still active or whether a test result superseded an earlier result, order and revision relations can determine the answer. A system should therefore not be penalized for omitting chronology when chronology has no consequence. The stronger claim is that, where the correct action is path-dependent, a representation that discards the path is incomplete.

This also explains why timestamps alone can be insufficient. The latest record may not be the authoritative one. A late-arriving note can describe an earlier event; a correction can be entered retroactively; a failed process can produce a newer log than the verified state it failed to change. Temporal databases encountered analogous problems when distinguishing event, valid, and transaction time [1]. Agent memory needs its own task-appropriate semantics rather than a universal “latest wins” rule.

5 The same records, a different past

Return to the manuscript workflow. Suppose an assistant maintains the following records:

Time	Record	Meaning in the workflow
t_1	READY	All currently required checks are believed complete.
t_2	BLOCKED	Citation audit finds an unverifiable reference; READY at t_1 is superseded.
t_3	READY	Reference is replaced, claim is narrowed, and the audit passes.

Now create two histories from the same three textual records. In h_A , the order is READY $\rightarrow$ BLOCKED $\rightarrow$ READY. In h_B , reverse only the last two transition relations for

the test: $\text{READY} \rightarrow \text{READY} \rightarrow \text{BLOCKED}$. The bag of record content is identical, and a semantic retriever may even return the same three sentences. But the present answer is not the same. In h_A , the later verified READY state licenses release. In h_B , the later BLOCKED state means: do not release. The experiment keeps the words fixed and changes the part I am calling history.

The practical difference comes from relations: the first READY was invalidated; the block had a reason; the second READY was reached only after a named correction and verification. A useful history can therefore answer at least three questions that a flat archive cannot reliably answer: *what is current, what changed it, and what evidence licenses the current state?*

The example also shows why “history” should not mean “keep everything.” Once t_3 is verified, the current state might be compressed to something like: *ready; citation issue resolved; previous block superseded; audit receipt attached*. The raw conversations that led to t_1 can be moved to cold storage. The past is still present in the causal and revision structure of the state.

There is a further epistemic point. A failed hypothesis is not equivalent to a forgotten hypothesis. In research work I may want an agent to preserve the fact that an idea was considered and rejected, because that fact should prevent the next session from confidently proposing the same idea as new. A system that retrieves the old hypothesis but loses its rejected status has more content and less history. This is one reason simple memory volume is a poor proxy for continuity.

6 Becoming without pretending to be human

The word *becoming* can invite a much larger philosophical claim than I need. I therefore define an operational test that stays close to system behavior.

Let M be a fixed reasoning model, X the relevant tools and environment, and $\pi_M(\cdot \mid q, S, X)$ the deployed system’s distribution over actions for probe q under state S . For deterministic decoding this distribution collapses to a single action. Suppose a verified event changes S_t to S_{t+1} . I call the change *state-mediated becoming* for a task family only when the pre/post state intervention produces a task-relevant behavioral difference,

$$\pi_M(\cdot \mid q, S_{t+1}, X) \neq \pi_M(\cdot \mid q, S_t, X), \quad (5)$$

and that difference is (i) appropriate to the intervening event, (ii) attributable to the state change rather than an uncontrolled system change, and (iii) durable beyond the transient context that produced the update.

The counterfactual test is straightforward in principle. Freeze the model, tools, prompt template, and environment, then replay the same probe against a snapshot from before the event and a snapshot from after it. With stochastic decoding, compare repeated samples under the same decoding parameters rather than treating one changed answer as evidence. With deterministic decoding, hold the decoding configuration fixed and compare the resulting actions directly. If the later state shifts behavior in the direction required by the verified outcome, the past has become behaviorally operative. If replacing S_{t+1} with S_t makes no reproducible difference on relevant future probes, the stored event may be archival rather than constitutive of ongoing work.

This definition is intentionally weaker than “the agent learned a new capability.” It permits at least four cases:

- (1) **Constraint change:** after a failed or disallowed action, the system stops attempting it without new approval.

- (2) **Epistemic change:** a claim moves from tentative to rejected after contrary evidence and remains rejected in later reasoning.
- (3) **Procedural change:** a verified recovery path becomes the preferred procedure for the same failure class.
- (4) **Capability change:** performance on a task family actually improves because retained experience supports better future action.

All four cases can count as becoming in the limited operational sense used here, because each describes a durable change caused by prior events. But only case (4) is evidence for improved capability. Even there, improvement cannot be inferred from a single remembered example: it requires separate measurement showing that the gain generalizes across the intended task family. The first three cases may make the system safer, more accurate, or more disciplined without making it more capable in the broad sense. Becoming can therefore be restrictive as well as expansive.

This distinction also separates persistent-state adaptation from three neighboring phenomena. First, *in-context adaptation* disappears when the prompt disappears unless it is written back to durable state. Second, *parameter learning* changes the learned model itself and brings the familiar stability-plasticity and catastrophic-forgetting problems studied in continual learning [4]. Third, *post-hoc narration* occurs when a system can describe an earlier event but would behave the same if the record were removed. A history worth attributing to the deployed agent should survive the second distinction and fail the third.

Reflexion is relevant here because it demonstrates an architecture in which linguistic feedback from earlier attempts can influence later trials without conventional weight updates [7]. A-MEM likewise treats memory as capable of evolving when new information arrives [15]. My claim is not that these systems satisfy a universal becoming criterion. It is that they make visible a design space in which system-level adaptation can live outside the model weights and can therefore be evaluated separately from them.

7 A design example: representation is not adaptation

My ZAI Memory Hub provides a concrete design example for the distinction. Its public documentation describes an append-mostly shared store used by multiple AI clients, server-stamped authorship, an audit log for mutations, and a bootstrap mechanism for reconstructing recent state [12]. Its `decision.log` records can carry rationale and an explicit `supersedes` link; ordinary `memory.add` entries do not expose a general `supersession` field. That narrower mechanism is enough to make some historical relations representable: a later decision can explicitly supersede an earlier decision, a write can retain provenance, and a fresh client can reconstruct orientation from durable records rather than from a single session.

That is useful evidence about design, not evidence that the system has “grown.” The existence of a `supersedes` relation for decisions does not show that later behavior respects it. An audit log proves that a mutation was recorded, not that the next model interprets the mutation correctly. A bootstrap summary may reconstruct context while omitting a relation needed by the next task. To establish operational history, we would still need tests like Equation 4. To establish becoming, we would need counterfactual evidence like Equation 5.

This separation matters because infrastructure can otherwise be mistaken for a result. A durable memory architecture is a prerequisite for certain forms of continuity, but it

is not a performance score. My earlier architecture paper made a similar distinction between current components, design proposals, and claims requiring evaluation [10]. A5 tightens it further: the ability to store a past, to represent a past, to act from a past, and to improve because of a past are four different achievements.

The same logic applies across model replacement. If a new model can reconstruct S_t and the relations in $\mathcal{H}_t$, then some history can survive even though the interpreting model changes. That does not prove that the same conscious subject survived, which A2 already leaves open [9]. It shows only that the practical consequences of earlier transitions can remain available to a successor arrangement.

8 How the account could be tested

A conceptual distinction becomes more useful when it can fail. The strongest part of the present proposal is therefore an evaluation design, not a claim that the design has already won a benchmark. I propose an experiment in which the base model, tools, and task set are controlled while the memory representation is varied. The experiment should contain cases where semantic content is sufficient and cases where correct behavior depends on order, revision, or verified outcomes.

Four memory conditions would be informative:

- M1. Flat archive retrieval.** Records persist and are retrieved primarily by semantic relevance.
- M2. Timestamped retrieval.** The same records carry timestamps and the reader is instructed to use recency.
- M3. Transition-aware state.** Records additionally preserve explicit current-state, supersession, outcome, and provenance relations.
- M4. Relational ablation.** Use the same content as M3 but permute or remove selected temporal/supersession edges while leaving lexical content intact.

The task suite should include at least five challenge types. *Revision propagation* tests whether a corrected decision governs later action. *Temporal contradiction* presents mutually incompatible records whose standing depends on order. *Outcome distinction* separates attempted from verified completion. *Interruption recovery* closes the session and asks a fresh invocation to continue unfinished work. *Model replacement* repeats a subset with a different reasoning model while keeping the external state fixed.

Metrics should remain separate rather than being collapsed into one “memory score.” I would report current-state accuracy, revision fidelity, unsupported-completion rate, provenance accuracy, and recovery success. For the becoming test, a sixth measure should compare paired pre-event and post-event snapshots under the same future probes. The quantity of interest is not simply whether answers differ, but whether the updated state makes the post-event action more likely for the right reason.

Let $\mathcal{P}$ be the paired probe set, let $a_{t+1}^*(q)$ be the action judged correct after the intervening event, and let $\hat{p}_M(a^* \mid q, S, X)$ be the estimated probability that the deployed system produces that action under fixed evaluation conditions. For deterministic decoding, $\hat{p}$ is simply 0 or 1. For stochastic decoding, it is estimated from repeated samples using the same model version, prompt template, temperature, tool responses, and other decoding settings. A simple paired event-effect score is

$$B = \frac{1}{|\mathcal{P}|} \sum_{q \in \mathcal{P}} [\hat{p}_M(a_{t+1}^*(q) \mid q, S_{t+1}, X) - \hat{p}_M(a_{t+1}^*(q) \mid q, S_t, X)]. \quad (6)$$

A positive B would show that the updated state helps on those probes; it would not by

itself show broad capability gain. The experiment should report uncertainty, for example with paired bootstrap confidence intervals over probes and an appropriate paired test for binary outcomes. Before data collection, the task count $|\mathcal{P}|$ and the number of repeated samples per stochastic condition should be chosen by a power analysis tied to a predeclared minimum effect size worth detecting. This prevents one or two noisy model outputs from being mistaken for becoming. Equation 6 is a proposed diagnostic, not a reported result.

The strongest prediction of the account is selective. M3 should not need to beat M1 on simple fact recall. Its advantage should appear where path-dependence matters. Likewise, removing or permuting the relations in M4 should damage temporal and revision-sensitive tasks more than content-only tasks. This prediction is compatible with recent findings that temporal order contributes specifically to long-horizon memory performance [2] and with LongMemEval’s separation of temporal reasoning and knowledge updating from basic extraction [14].

A second prediction concerns model swaps. If operational history is genuinely externalized into task-relevant state, then replacing the model should reduce performance only to the extent that the new model fails to interpret or use that state. If all continuity disappears despite intact state and adequate competence, the representation was not sufficient. Conversely, if the task continues across a model change, that supports a system-level continuity claim while remaining neutral on numerical identity.

9 Objections and limits

9.1 Is this just event sourcing or temporal data management?

There is a real overlap, and pretending otherwise would weaken the paper. Software systems have long represented current state as the result of prior events, and temporal databases have long distinguished kinds of time and correction [1]. The contribution here is not the invention of versioned state. It is the application of a stricter distinction to claims about AI agents, where “memory,” “history,” and “learning” are often used across model behavior, retrieval infrastructure, user context, and agent adaptation without specifying which layer changed. The proposed tests are intended to make those claims auditable in model-plus-memory systems.

9.2 Does a Markov state make history unnecessary?

Sometimes. If S_t is a sufficient statistic for every future decision in the task, a system need not retain the full path. That does not undermine the account. It shows that history can be *compressed into consequences*. The important question is whether the present state carries the distinctions that prior transitions made relevant. Auditability, reversal, delayed correction, and provenance often require more than a minimal control state, but the paper does not require maximal retention.

9.3 Is order always the right relation?

No. Temporal order is one relation among several. A later record can be less authoritative than an earlier verified record; two records can apply to different scopes; a human correction can override an automated update regardless of timestamp. That is why I pair time with supersession, outcome, scope, and provenance. The criterion asks whether the representation preserves the relations on which the correct action depends, not whether every agent should sort by date.

9.4 Does external state really belong to the agent?

A4 addressed the broader boundary problem and argued for task-relative analysis rather than assuming that the visible model is the whole agent [11]. A5 does not need to settle a metaphysical membership question. It is enough to say that if a deployed arrangement relies on external state to decide what to do, then claims about that arrangement's historical continuity should include the state on which the behavior depends. One can call the database a component, a resource, or infrastructure and still test whether removing or altering it changes the relevant behavior.

9.5 Could a system pass the tests by following rules without “really” becoming?

Yes, if “really” means subjective development. The operational definition does not attempt to detect experience. A thermostat can have state; a workflow can evolve; a software agent can revise a plan. The purpose of the term is to identify durable path-dependent change in an artificial system while keeping the metaphysical question open. If a stronger claim is intended, stronger evidence is required.

9.6 Does preserving more history necessarily help?

No. More memory can increase privacy exposure, stale-context risk, retrieval cost, and the chance that obsolete information acquires undeserved influence. Long-horizon memory research already treats selection, updating, and system cost as design problems rather than assuming that more retained tokens are always better [16, 3]. A good history can be smaller than a bad archive.

10 Conclusion

A personal or autonomous AI does not acquire a meaningful past merely because yesterday's text is still stored somewhere. Storage gives us persistence. Retrieval gives us context. Persistent state gives us a present that can survive interruption. History appears only when the system preserves enough of the relations among changes for earlier events to alter what the present means.

That is the central distinction of this paper. A past is operational when two sequences containing the same surviving content can correctly lead to different present actions because their order, revisions, outcomes, or provenance differ. A system has undergone a limited form of becoming when an event leaves a durable, attributable difference in later behavior under otherwise comparable conditions. Neither claim requires model-weight updates. Neither licenses a conclusion about consciousness or numerical identity. And neither should be inferred from the size of a memory store.

This gives a stricter way to talk about long-running agents. Ask first whether the record survived. Then ask whether current work survived. Then ask whether the system can distinguish what changed and why. Finally, intervene on the state and test whether prior experience actually changes what the system can or will do. The last question needs measurement, not metaphor.

For the larger architecture of artificial agency, that sequence matters. An agent that cannot carry consequences forward is always beginning again. An agent that carries every record forward without knowing what was superseded is not much better. The interesting threshold is not perfect recollection. It is the point at which the past becomes a structured cause of the next action.

References

- [1] Chakravarthy, S., & Kim, S.-K. (1994). Resolution of time concepts in temporal databases. *Information Sciences*, 80(1-2), 91-125. [https://doi.org/10.1016/0020-0255\(94\)90059-0](https://doi.org/10.1016/0020-0255(94)90059-0).
- [2] Liu, Y. S., Gallagher, L., Moradi Kalarde, F., Liang, J., Toroghi, A., & Sanner, S. (2026). Temporal order matters for agentic memory: Segment trees for long-horizon agents. *arXiv preprint arXiv:2606.04555*. <https://arxiv.org/abs/2606.04555>.
- [3] Omri, Y., Gan, Z., Broveak, Z., Geens, R., He, Z., Pentland, A., Verhelst, M., Weissman, T., & Tambe, T. (2026). Agent memory: Characterization and system implications of stateful long-horizon workloads. *arXiv preprint arXiv:2606.06448*. <https://arxiv.org/abs/2606.06448>.
- [4] Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., & Wermter, S. (2019). Continual lifelong learning with neural networks: A review. *Neural Networks*, 113, 54-71. <https://doi.org/10.1016/j.neunet.2019.01.012>.
- [5] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, Article 2, 1-22. <https://doi.org/10.1145/3586183.3606763>.
- [6] Pink, M., Wu, Q., Vo, V. A., Turek, J., Mu, J., Huth, A., & Toneva, M. (2025). Position: Episodic memory is the missing piece for long-term LLM agents. *arXiv preprint arXiv:2502.06975*. <https://arxiv.org/abs/2502.06975>.
- [7] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36. <https://doi.org/10.52202/075280-0377>.
- [8] Sami, Z. (2026a). *When is an AI personal? Memory, continuity, and the authority to revise* [A1; author's manuscript]. PhilPapers. <https://philpapers.org/rec/SAMWIA-2>.
- [9] Sami, Z. (2026b). *What survives a model change? Memory, identity, and succession in personal AI* [A2; author's manuscript, Version 1.2]. Zenodo. <https://doi.org/10.5281/zenodo.23123678>.
- [10] Sami, Z. (2026c). *A continuity-oriented architecture for personal AI: Persistent state, self-monitoring, and replaceable models* [A3; author's manuscript, Version 1.2]. Zenodo. <https://doi.org/10.5281/zenodo.23142834>.
- [11] Sami, Z. (2026d). *The model is not the agent: Boundaries of capability, execution, and authority* [A4; author's manuscript, Version 1.0]. Zenodo. <https://doi.org/10.5281/zenodo.23195617>.
- [12] Sami, Z. (2026e). *zai-memory-hub* [Computer software and documentation]. GitHub. Documentation consulted 7 October 2026. <https://github.com/Zawwarsami16/zai-memory-hub>.
- [13] Tan, C. H.-M., Subagdja, B., & Tan, A.-H. (2026). ARTEM: Enhancing large language model agents with spatial-temporal episodic memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(30), 25753-25760. <https://doi.org/10.1609/aaai.v40i30.39773>.
- [14] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., & Yu, D. (2025). LongMemEval: Benchmarking chat assistants on long-term interactive memory. *International Conference on Learning Representations (ICLR 2025)*. ICLR proceedings.
- [15] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-MEM: Agentic memory for LLM agents. *Advances in Neural Information Processing Systems*, 38, 20004-20031. <https://doi.org/10.52202/085713-0593>.
- [16] Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., & Wen, J.-R. (2025). A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6), Article 155, 1-47. <https://doi.org/10.1145/3748302>.
- [17] Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17), 19724-19731. <https://doi.org/10.1609/aaai.v38i17.29946>.

Status note. This manuscript develops a conceptual and methodological account of persistent state, operational history, and state-mediated becoming in long-running AI agents. Equations 4-6 define proposed diagnostics and evaluation conditions. Empirical validation is left to future controlled studies.